(Task 1 of 15) In this tutorial, we will learn *more* about definitions. 0

Pseudo | 1

(Task 2 of 15) What is the result of running this program?

Pseudo [Run ▶]

```
let x = 1
fun f(y):
  fun g():
    let z = 2
    return x + y + z
  end
  return g()
end
print(f(3) + 4)
```

Scala 3

```
val x = 1
def f(y : Int) =
  def g =
    val z = 2
    x + y + z
  g
println(f(3) + 4)
```

`10` 2

Please briefly explain why you think the answer is `10`. 3

x = 1, z = 2, y = 3, so 1 + 3 + 2 is 6, then 6 + 4 is 10 4

You predicted the output correctly 🎉 🎉 🎉 5

This program binds `x` to `1` and `f` to a function and then evaluates `f(3) + 4`. The value of `f(3) + 4` is the value of `g() + 4`, which is the value of `(x + y + z) + 4`, which is the value of `(1 + 3 + 2) + 4`, which is `10`.

Click here to run this program in the Stacker.

Pseudo | 6

(Task 3 of 15) What is the result of running this program?

Pseudo [Run ▶]

```
let x = 1
fun f(y):
  let x = 2
  return x + y
end
print(f(0) + x)
```

Python

```
x = 1
def f(y):
    x = 2
    return x + y
print(f(0) + x)
```

`3` 7

You predicted the output correctly 🎉 🎉 🎉 8

This program binds `x` to `1` and `f` to a function in the top-level block. It binds `x` to `2` in the body of `f`. So, `f(0)` evaluates to `2 + 0`, which is `2`, and `f(0) + x` evaluates to `2 + 1`, which is `3`.

Click here to run this program in the Stacker.

Pseudo | 🎲

(Task 4 of 15) What is the result of running this program?

Pseudo [Run ▶]

```
let x = 1
fun f():
  return x
end
fun g():
  let x = 2
  return f()
end
print(g())
```

Scala 3

```
val x = 1
def f =
  x
def g =
  val x = 2
  f
println(g)
```

`1`

10

11

You predicted the output correctly 🎉 🎉 🎉

`g()` evaluates to `f()`, which evaluates to `1` because `f` and `x` are both defined in the same block, and `f` does *not* get the `x` defined inside `g`.

Click here to run this program in the Stacker.

Pseudo | 🎲

(Task 5 of 15) What is the result of running this program?

Pseudo [Run ▶]

```
fun f(x):
  let y = 1
  return x + y
end
print(f(2) + y)
```

Python

```
def f(x):
    y = 1
    return x + y
print(f(2) + y)
```

`4`

13

14

The answer is `error`.

▼ Textual explanation

`let y = 1` binds `y` to `1`. You might think the binding applies everywhere. However, it only applies to the body of `g`. `f(2) + y` appears in the top-level block, so it tries to refer to a `y` defined in the top-level block. But `y` is not defined in the top-level block. In SMoL, variable references follow the hierarchical structure of blocks.

Click here to run this program in the Stacker.

Pseudo | 🎲

What is the result of running this program?

Pseudo [Run ▶] Scala 3

```
fun foo(bar):
  let zzz = 8
  return bar * zzz
end
print(foo(9) * zzz)
```

```
def foo(bar : Int) =
  val zzz = 8
  bar * zzz
println(foo(9) * zzz)
```

`error`

16

You predicted the output correctly 🎉 🎉 🎉

17

Pseudo | 🎲

(Task 6 of 15) What is the result of running this program?

Pseudo [Run ▶] Python

```
let x = 1
fun f():
  let y = 2
  fun g():
    return x + y
  end
  return g()
end
print(f())
```

```
x = 1
def f():
    y = 2
    def g():
        return x + y
    return g()
print(f())
```

`3`

19

Please briefly explain why you think the answer is `3`.

20

x = 1, y = 2, 2 + 1 = 3, deffun (f) return g which is 3

21

You predicted the output correctly 🎉 🎉 🎉

22

`f()` returns the value of `g()`. The value of `g()` is the value of `x + y`, which is `1 + 2`, which is `3`.

Click here to run this program in the Stacker.

(Task 7 of 15) When we see a variable reference (e.g., the `x` in `x + 1`), how do we find its value, if any? 23

if it is declared in the scope 24

(Task 8 of 15) Variable references follow the hierarchical structure of blocks. 25

If the variable is defined in the current block, we use that declaration.

Otherwise, we look up the block in which the current block appears, and so on recursively. (Specifically, if the current block is a function body, the next block will be the block in which the function definition is; if the current block is the top-level block, the next block will be the **primordial block**.)

If the current block is already the primordial block and we still haven't found a corresponding declaration, the variable reference errors.

The primordial block is a non-visible block enclosing the top-level block. This block defines values and functions that are provided by the language itself.

Any feedback regarding these statements? Feel free to skip this question. 26

(You skipped the question.) 27

(Task 9 of 15) Now please scroll back and select 1-3 programs that make the point that 28

> If the variable is declared in the current block, we use that declaration.

You don't need to select *all* such programs.

(*You selected 3 programs*) 29

Okay. How do these programs (1,6,9) support the point? 30

because it can be used 31

(Task 10 of 15) Please select 1-3 programs that make the point that 32

> Otherwise, we look up the block in which the current block appears, and so on recursively.

You don't need to select *all* such programs.

(*You selected 2 programs*) 33

Okay. How do these programs (6,18) support the point? 34

it appear outside the block 35

(Task 11 of 15) Please select 1-3 programs that make the point that 36

> If the current block is already the primordial block and we still haven't found a corresponding declaration, the variable reference errors.

You don't need to select *all* such programs.

(*You selected 2 programs*) 37

Okay. How do these programs (12,15) support the point? 38

because it wasn't declare so it lead to error 39

(Task 12 of 15) The **scope** of a variable is the region of a program where we can refer to the variable. Technically, it includes the block in which the variable is defined (including sub-blocks) *except* the sub-blocks where the same name is re-defined. When the exception happens, that is, when the same name is defined in a sub-block, we say that the variable in the sub-block **shadows** the variable in the outer block. 40

We say a variable reference (e.g., "the `x` in `x + 1`") is **in the scope of** a declaration (e.g., "the `x` in `let x = 23`") if and only if the former refers to the latter.

Any feedback regarding these statements? Feel free to skip this question. 41

(You skipped the question.) 42

(Task 13 of 15) Please select all statements that apply to the following program: 43

```
let x = 45
fun f():
  fun g():
```

```
    return x + 1
  end
  let x = 67
  return x + 2
end
print(f())
print(x + 3)
```

44

- [ ] The scope of the top-level `x` includes the whole program.
- [x] The scope of the top-level `x` includes `x + 3`.
- [ ] The top-level `x` shadows the `x` defined in `f`.
- [ ] The `x` in `x + 1` is in the scope of the `x` defined in `f`.
- [x] The `x` in `x + 2` is in the scope of the `x` defined in `f`.
- [ ] The `x` in `x + 3` is in the scope of the `x` defined in `f`.

45

You should also have chosen The `x` in `x + 1` is in the scope of the `x` defined in `f`.

The scope of the top-level `x` includes the whole program *except* the body of `f`. The scope of the `x` defined in `f` is the body of `f`.

The `x` defined in `f` shadows the top-level `x` rather than the other way around.

Pseudo | 🎲

(Task 14 of 15) Here is a program that confused many students

Pseudo [Run ▶] Lispy

```
let x = 1
fun foo():
  return x
end
fun bar():
  let x = 2
  return foo()
end
print(bar())
```

```
(defvar x 1)
(deffun (foo)
  x)
(deffun (bar)
  (defvar x 2)
  (foo))

(bar)
```

Please

1. Run this program in the stacker by clicking one of the [Run ▶] buttons above;
2. The stacker would show how this program produces its result(s);
3. Keep clicking ⏭ Next until you reach a configuration that you find particularly helpful;
4. Click 🔗 Share This Configuration to get a link to your configuration;
5. Submit your link below;

https://smol-tutor.xyz/stacker/?syntax=Pseudo&randomSeed=smol-tutor&hole=%E2%80%A2&nNext=6&program=%0A%28defvar+x+1%29%0A%28deffun+%28foo%

47

(Task 15 of 15) Please write a couple of sentences to explain how your configuration explains the result(s) of the program.

48

so it check function body and it bind the number into it own variable, doing it for all function until it solve the output

49

Let's review what we have learned in this tutorial.

50

Variable references follow the hierarchical structure of blocks.

If the variable is defined in the current block, we use that declaration.

Otherwise, we look up the block in which the current block appears, and so on recursively. (Specifically, if the current block is a function body, the next block will be the block in which the function definition is; if the current block is the top-level block, the next block will be the **primordial block**.)

If the current block is already the primordial block and we still haven't found a corresponding declaration, the variable reference errors.

The primordial block is a non-visible block enclosing the top-level block. This block defines values and functions that are provided by the language itself.

The **scope** of a variable is the region of a program where we can refer to the variable. Technically, it includes the block in which the variable is defined (including sub-blocks) *except* the sub-blocks where the same name is re-defined. When the exception happens, that is, when the same name is defined in a sub-block, we say that the variable in the sub-block **shadows** the variable in the outer block.

We say a variable reference (e.g., "the `x` in `x + 1`") is **in the scope of** a declaration (e.g., "the `x` in `let x = 23`") if and only if the former refers to the latter.

You have finished this tutorial 🎉 🎉 🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1775700073000